

COrtai V1.0

Single-EC2 Deployment Specification

Hotel Operations Platform : Pilot Architecture

TownePlace Suites Marriott Vaughan (122 rooms)

V1.0 deadline: September 1, 2026

Prepared by: Yassine Essahli

Date: 2026-05-01

Table of Contents

1. Executive Summary
2. Available Infrastructure (provided by Lionston)
3. V1.0 Global Architecture
4. Inside the EC2 : Service Map
5. Service Catalog
6. Critical Data Flows
7. Network & VPN Access
8. Database : RDS PostgreSQL 17.6
9. TLS & DNS Strategy (port 80 closed)
10. Scaling Roadmap V1.0 → V3.0

1. Executive Summary

This document specifies the V1.0 pilot deployment of the CORTai Unified hotel operations platform on the AWS infrastructure already provisioned by Lionston Innovations. The deployment runs on a single EC2 t3.medium instance accessible via the LS-VPN, backed by a managed RDS PostgreSQL 17.6 database and an S3 bucket. It is sized to support up to 10 hotels with a total of 100 BST A1000 edge devices ingesting in real time,. The architecture is deliberately small and pragmatic: a modular monolith FastAPI service for the business logic, three lightweight Go services for high-throughput edge ingestion, and managed AWS components for storage and AI.

Design principles:

- Start small: one EC2 with all services as systemd units. No Kubernetes for V1.0.
- Modular monolith: business logic (PMS, housekeeping, forms, alerts, AI) in a single FastAPI process with Python module boundaries that mirror the long-term microservice split.
- Three dedicated Go services: edge-uplink (WebSocket terminator for 100 A1000s), ingest-worker (batch INSERT to TimescaleDB), and ingest-only paths kept off the FastAPI critical path.
- Managed external services: RDS for Postgres, S3 for objects, Bedrock for LLM, Anthropic for embeddings. No self-managed databases or model servers.
- VPN-only access: the EC2 is not publicly exposed. All clients reach it through the LS-VPN. A1000s in production hotels will reach it through site-to-site VPN tunnels.
- Direct WebRTC for video: the A1000s and viewers are inside the same VPN, so STUN suffices and Coturn TURN is deferred to V2.0.
- Clear scaling path: every service boundary is process-level, so each can be extracted to its own EC2 or ECS task in 1-2 days when load demands it.

2. Available Infrastructure (provided by Lionston)

The Lionston network engineering team has provisioned the following AWS resources in the ca-central-1 region:

Resource	Specification	Access
EC2 instance	t3.medium · 2 vCPU · 4 GB RAM · ~30 USD/month	ec2-35-183-30-14.ca-central-1.compute.amazonaws.com — SSH via LS-VPN or PandoraBox jump host
RDS PostgreSQL	Engine 17.6 · db.t3.small · 1 vCPU · 2 GB RAM	ls-videosurveillance-db.cxqq080ygcgn.ca-central-1.rds.amazonaws.com:5432 — VPC private, EC2 only
S3 bucket	Standard storage class	s3://ls-videosurveillance-s3 — accessible from EC2 only
Open ports (EC2)	22 (SSH), 443 (HTTPS), 5000, 5001, 8080	All inbound limited to LS-VPN range
Closed ports	80 (intentionally not opened)	Use 8080 for plain HTTP
Database credentials	user=postgres / db=postgres	Provided in onboarding email
Network access	LS-VPN site-to-site	Reaches the lab subnet 172.16.100.0/24 and the EC2

3. V1.0 Global Architecture

The platform is organised in three tiers : Edge (BST A1000 devices in hotels), Cloud (single EC2 in AWS), and Clients (web, mobile, signage). All edge-to-cloud traffic flows through the LS-VPN. All client-to-cloud traffic uses HTTPS on port 443. Video streams from A1000 to browser are direct WebRTC peer-to-peer connections inside the VPN, they do not transit through the EC2.

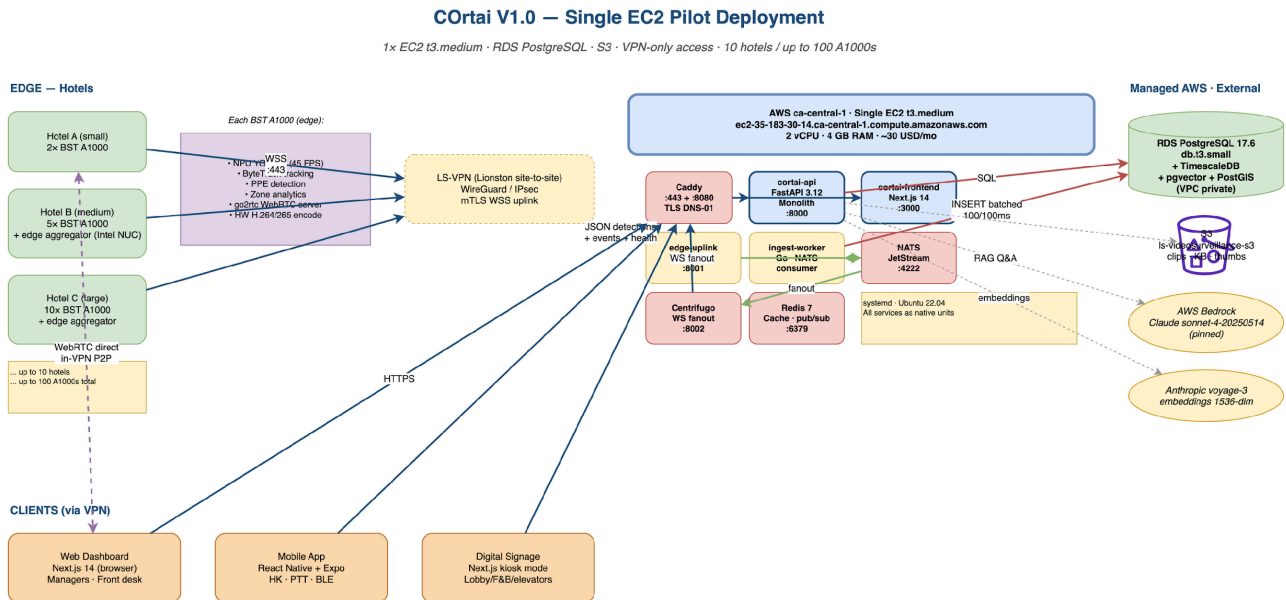


Diagram 1: V1.0 Global Architecture

Tier overview:

- **EDGE per hotel:** Each BST A1000 runs the existing computer-vision pipeline (HW H.264/265 decode, NPU YOLOv8 detection at 45 FPS, ByteTrack tracking, PPE detection, zone analytics, HW encode). For hotels with 4 or more A1000s, an Intel NUC edge aggregator provides local Mosquitto, Redis cache, SQLite pool for offline operation, and a single multiplexed WSS uplink to the cloud.
- **CLOUD:** A single EC2 t3.medium runs Caddy (TLS, reverse proxy), the FastAPI monolith (cortai-api), the Next.js frontend, three Go services (edge-uplink, ingest-worker), NATS JetStream solo, Centrifugo, and Redis, all as systemd units. The RDS PostgreSQL 17.6 instance hosts the full schema with TimescaleDB, pgvector, and PostGIS extensions. S3 stores video clips, knowledge-base documents, and signage assets.
- **CLIENTS:** Browser-based Next.js dashboard for managers and front desk; React Native mobile application for housekeeping, PTT, and BLE positioning; Next.js kiosk-mode digital signage. All clients connect through the LS-VPN.

4. Inside the EC2 : Service Map

All processes run as native systemd units on Ubuntu 22.04. No containers in V1.0 : direct binaries simplify operations and reduce overhead on a 4 GB RAM instance. The total committed memory is approximately 1.5 GB, leaving 2.5 GB of margin for spikes, page cache, and growth. Average CPU usage is around 30 % with peaks up to 70 %, well within the burstable credit envelope of t3.medium for the V1.0 load.

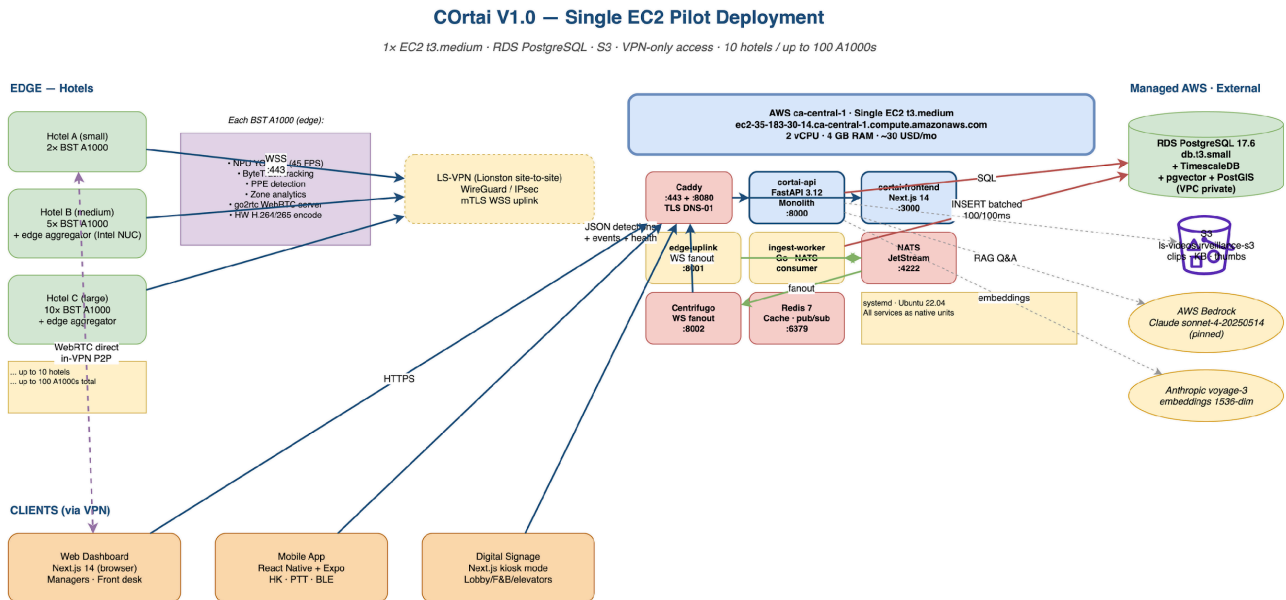


Diagram 2 : Service Map Inside the EC2

5. Service Catalog

The platform consists of nine services running on the EC2. Each is documented below with its purpose, port, and approximate resource footprint.

Service	Tech	Port	RAM	Purpose
caddy.service	Caddy 2.7 (Go)	443, 8080	~50 MB	TLS termination (Let's Encrypt DNS-01), reverse proxy to all internal services, OWASP CRS WAF, per-tenant rate limit
cortai-api.service	FastAPI 3.12 + uvicorn	8000	~600 MB	Modular monolith, pms, housekeeping, forms, alerts, revenue, hr, admin, ai modules. RLS multi-tenant via SET LOCAL app.current_org_id.
cortai-frontend.service	Next.js 14 standalone (Node 20)	3000	~300 MB	Web dashboard. App Router with React Server Components and streaming SSR.
edge-uplink.service	Go 1.22 (gorilla/websocket)	8001	~80 MB	Terminates persistent WSS connections from up to 100 BST A1000 devices. Validates JWT per device. Publishes to NATS topics.
ingest-worker.service	Go 1.22 (NATS subscriber)	—	~80 MB	Subscribes to detection topics, batches inserts (100 rows / 100 ms) into TimescaleDB hypertables, runs declarative anomaly rules, forwards events to Centrifugo.
nats-server.service	NATS JetStream (single node)	4222, 8222	~150 MB	Persistent message bus with 24-hour replay. Topics: detections.*, events.*, health.*, alerts.*
centrifugo.service	Centrifugo OSS (Go)	8002	~80 MB	WebSocket fanout to dashboards and mobile apps. JWT-authenticated channels scoped to org_id.
redis-server.service	Redis 7	6379	~150 MB	Cache for RAG responses, per-IP rate limiting, pub/sub backup channel, LangGraph state checkpoints.
postgresql (RDS)	PostgreSQL 17.6 + TimescaleDB + pgvector + PostGIS	5432	managed	Single source of truth. Multi-tenant via row-level security on org_id.

6. Critical Data Flows

Four flows define the runtime behaviour of the platform: real-time detection ingestion, RAG question answering, direct video peering, and IoT-driven alerting.

Inside the EC2 t3.medium — Service Map

All processes as systemd units · ~1.5 GB RAM committed of 4 GB · ~6-8 cores peak of 2 (burstable)

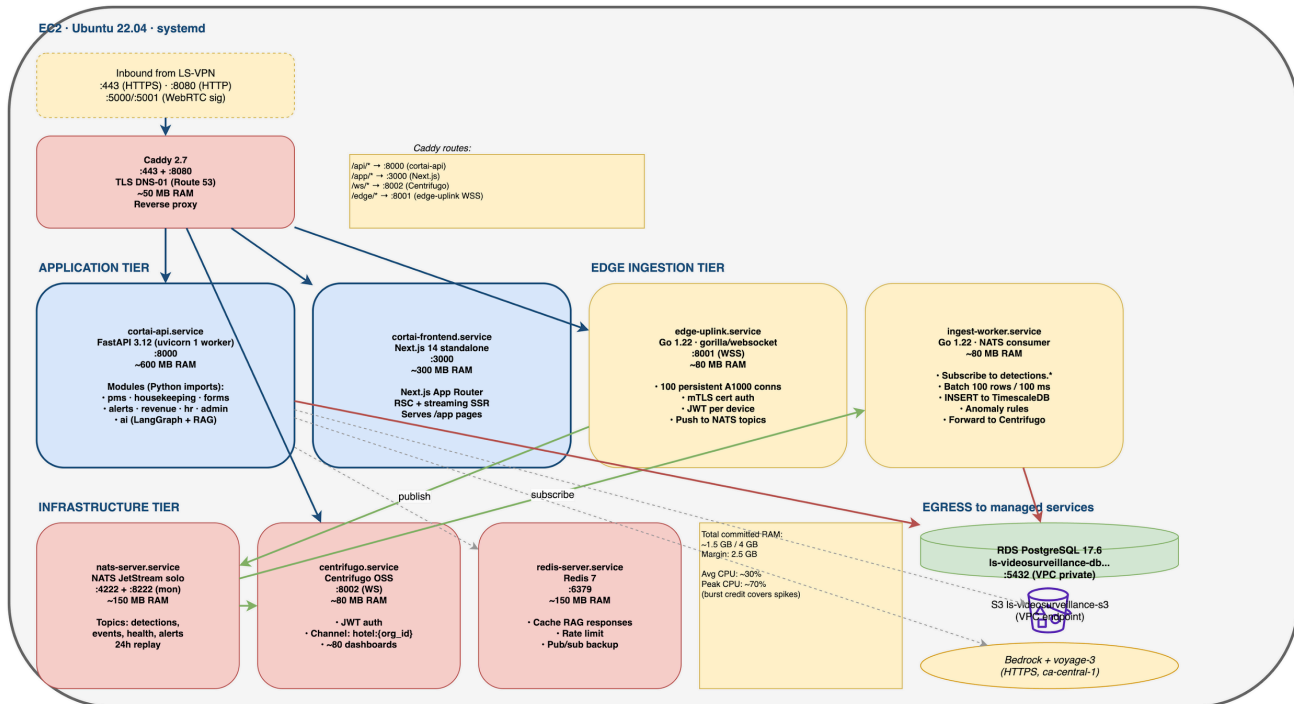


Diagram 3 : Critical Data Flows

6.1 Camera detection → live dashboard

- Each BST A1000 produces JSON detections (bounding boxes, tracking IDs, confidence scores) at up to 45 FPS per camera
- The A1000 batches detections every 100 ms and pushes them to edge-uplink over a persistent WSS connection
- edge-uplink validates the JWT, attaches org_id and property_id, and publishes to the NATS subject detections.<org>.<property>.<camera>
- ingest-worker subscribes, batches up to 100 rows per 100 ms window, and INSERTs into iot.camera_detections (TimescaleDB hypertable, 90-day retention)
- ingest-worker also forwards a compact event to Centrifugo, which fans out to dashboards subscribed to channel hotel:<org_id>
- Browser draws the bounding boxes as canvas overlays on the live <video> element

6.2 Cortai Intel — RAG Q&A

- Manager types a question in the dashboard; the browser opens a Server-Sent Events stream to /api/ai/query
- cortai-api's ai/ module records the conversation in ai.llm_conversations and the user message in ai.llm_messages

- The active prompt template is loaded from `ai.prompt_templates` (status active or canary)
- The tool catalogue is loaded from `ai.function_tools` (status active, scoped to the user's role)
- The question is embedded with Anthropic voyage-3 (1536 dimensions)
- An ANN search runs against `ai.vector_embeddings` (HNSW index, `ef_search = 40`), retrieving the top-K relevant chunks
- Bedrock Claude (`claude-sonnet-4-20250514`, pinned) is invoked with the retrieved context and tool catalogue
- Tool calls execute under the `cortai_llm` role with `SET LOCAL app.current_org_id`, enforcing row-level security
- Tokens are streamed back to the browser as Server-Sent Events
- The assistant message is recorded in `ai.llm_messages` with `retrieved_doc_ids[]` for full citation audit

6.3 Direct WebRTC video : no Coturn in V1.0

- Manager clicks a camera tile; the browser issues `GET /api/cameras/{id}/stream-url`
- `cortai-api` returns the in-VPN URL of the A1000's go2rtc signalling endpoint, `ws://<a1000-private-ip>:5000/api/ws`
- The browser opens an `RTCPeerConnection` directly to the A1000
- STUN suffices for NAT traversal because both ends share the LS-VPN routing fabric
- Video flows peer-to-peer at sub-100 ms latency with no relay through the EC2
- Coturn (TURN relay) is deferred until V2.0, when public-internet viewers (off-VPN) need to view streams

6.4 IoT sensor → alert

- BLE beacons, door contacts, occupancy sensors, HVAC thermostats, water-leak sensors publish to the local Mosquitto broker on the edge aggregator
- edge-uplink bridges MQTT to NATS topic alerts.<org>.<property>.<sensor_class>
- `cortai-api`'s alerts module evaluates declarative YAML rules against the incoming events
- Threshold crossings are written to `iot.alerts` and `ai.anomaly_detections`
- Centrifugo broadcasts the alert to dashboards; FCM/APNS push notifications go to mobile app users on shift
- If severity = critical, forms-service auto-creates an incident record

7. Network & VPN Access

The EC2 has no public-internet exposure. All access SSH, HTTPS, edge uplink, dashboard browsing, flows through the Lionston LS-VPN. This is a deliberate security choice for the pilot phase. Each production hotel will require a dedicated site-to-site VPN tunnel between the hotel's Internet gateway and the Lionston VPN concentrator. The site-to-site provisioning workflow is owned by Lionston Network Engineering.

Port plan:

Port	Direction	Protocol	Purpose
22/tcp	Inbound (VPN)	SSH	Administration, restricted to Yassine's authorized public key
443/tcp	Inbound (VPN)	HTTPS	Web app, REST API, WebSocket upgrade for Centrifugo, edge-uplink WSS multiplexed under /edge/*
8080/tcp	Inbound (VPN)	HTTP	Health checks, internal admin endpoints (no public traffic)
5000/tcp	Outbound (VPN)	WS	WebRTC signalling — browser → A1000 (direct, not via EC2)
5001/tcp	Outbound (VPN)	RTSP	Camera RTSP — A1000 → cameras (intra-LAN at hotel)
5432/tcp	Outbound (VPC)	PostgreSQL	EC2 → RDS, private VPC routing
443/tcp	Outbound (Internet)	HTTPS	EC2 → Bedrock + voyage-3 + S3 endpoint

8. Database: RDS PostgreSQL 17.6

The platform uses existing 18-file SQL schema (8 namespaced schemas: platform, hospitality, people, iot, audit, revenue, analytics, ai). Three Postgres extensions are required and must be enabled before applying the schema.

Extension	Required for	Action
pgvector	ai.vector_embeddings (1536-dim voyage-3 vectors with HNSW index)	CREATE EXTENSION vector; available by default on RDS PG 17
postgis	platform.properties.geo_point, BLE positioning, geofencing	CREATE EXTENSION postgis;
timescaledb	iot.camera_detections, iot.sensor_readings, iot.staff_positions hypertables	Requires shared_preload_libraries to include timescaledb in the RDS parameter group, coordinate with Network Engineering

Database roles (created by 00_setup.sql):

- cortai_app : read/write application role used by cortai-api and ingest-worker
- cortai_llm : SELECT-only role bound to RLS, used by the langgraph-orchestrator path inside cortai-api
- cortai_readonly : analytics and BI
- cortai_ingest : write-only on iot tables, used by edge-uplink

9. TLS & DNS Strategy (port 80 closed)

Because port 80 is intentionally closed at the security group level, the standard Let's Encrypt HTTP-01 challenge is unavailable. Three options exist; option (a) is recommended because it is fully automated and renews silently.

- (a) DNS-01 challenge with Caddy + Route 53 plugin : Caddy runs the DNS-01 ACME challenge against the Lionston-controlled DNS zone. Requires an IAM access key with `route53:ChangeResourceRecordSets` and `route53:GetChange` on a single hosted zone.
- (b) Manually provisioned wildcard certificate : `*.cortai.lionston.com` issued by Lionston's CA or DigiCert/Sectigo. The certificate file is placed at `/etc/caddy/certs/wildcard.pem` and renewed annually by Network Engineering.
- (c) Self-signed certificate : only for closed-network smoke tests. Browsers will warn. Avoid for stakeholder demos.

DNS records to create (in the Lionston zone):

- `api.cortai.lionston.com` → EC2 elastic IP (REST + WSS via Caddy)
- `app.cortai.lionston.com` → EC2 elastic IP (Next.js dashboard)
- `ws.cortai.lionston.com` → EC2 elastic IP (Centrifugo)
- `edge.cortai.lionston.com` → EC2 elastic IP (edge-uplink WSS)

10. Scaling Roadmap V1.0 → V3.0

The V1.0 architecture is intentionally minimal but the service boundaries match the long-term V3.0 microservices design. Each future split is therefore mechanical : extract a process, point Caddy or NATS at it, deploy on a separate instance. Scale only on observed signal, never in anticipation.

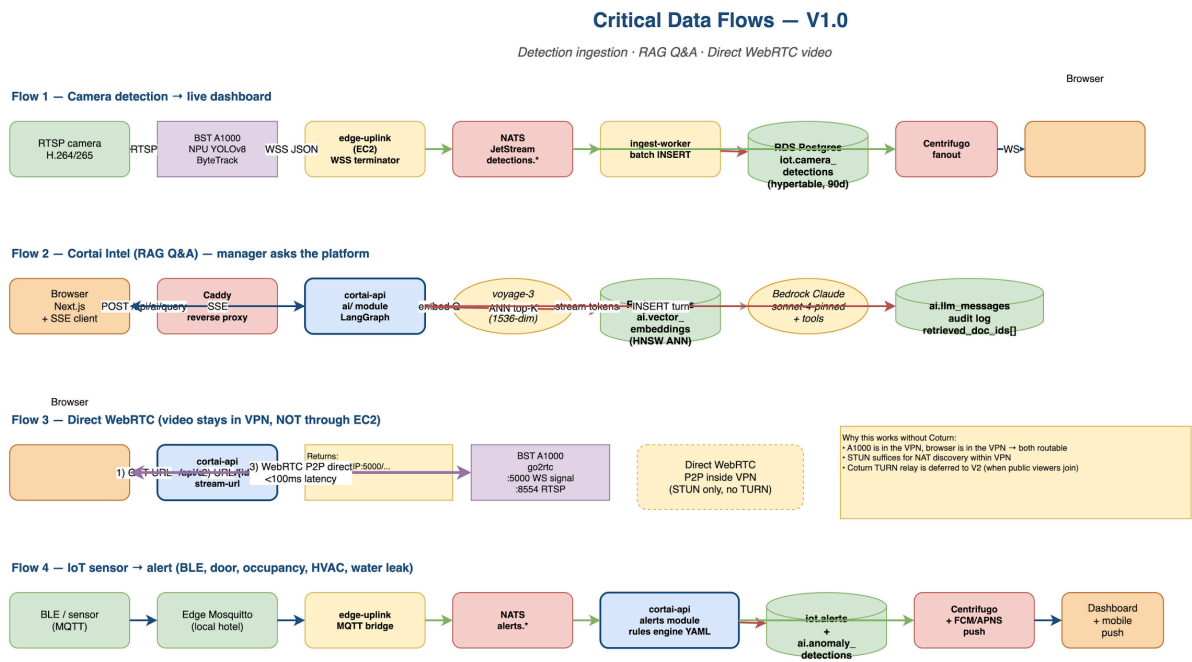


Diagram 4 : Roadmap and Scaling Triggers